# PROG 2B ASSIGNMENT 1

ST10467898

| HTTP method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| POST | /api/auth/register | Registers a new RaceDay user as an organizer or participant | None(public) | {fullName, email, password, role} | 201 created-account created. 400 bad request-invalid data. 409 conflict-email already exists |
| POST | /api/auth/login | Logs a registered user into the RaceDay system | None(public) | {email, password} | 200 OK - login successful. 401 Unauthorized - invalid credentials. |
| POST | /api/auth/logout | Logs the current user out of the system | Any(logged in) | None | 204 No Content - logout successful. 401 Unauthorized - user not logged in. |
| GET | /api/profile | Displays the logged in users profile information | Any(logged in) | None | 200 OK - profile returned. 401 Unauthorized - user not logged in. |
| PUT | /api/profile | Updates the logged in | Any(logged in) | {fullName, phoneNumber, dateOfBirth, | 200 OK - profile updated. |

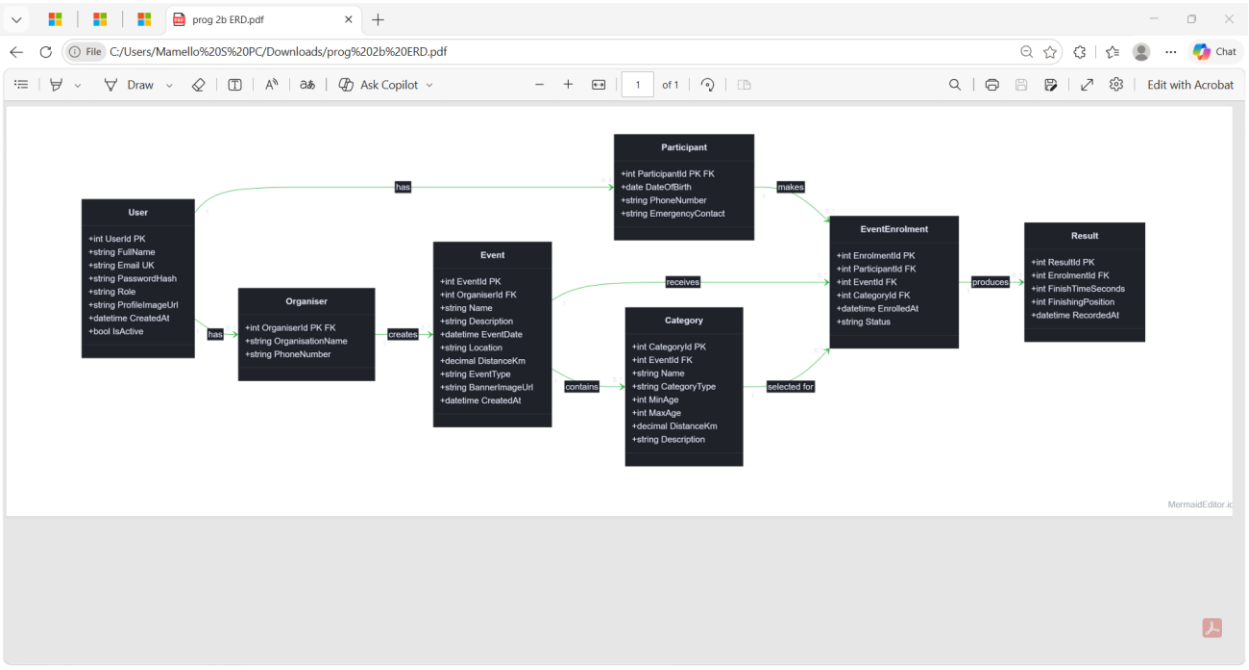
| | | | | | |
|---|---|---|---|---|---|
| | | users profile information | | emergencyContact} | 400 Bad Request - invalid data. |
| GET | /api/events | Displays all RaceDay events | Any(logged in) | None | 200 OK - list of events returned. |
| GET | /api/events/{eventId} | Displays the full information for a selected event | Any(logged in) | None | 200 OK - event returned. 404 Not Found - event does not exist. |
| GET | /api/organizer/events | Displays all the events created by the logged in organizer | organizer | None | 200 OK - Organiser's events returned. 403 Forbidden - incorrect role. |
| POST | /api/events | Creates new event | organizer | {name, description, eventDate, location, distanceKm, eventType, bannerImageUrl} | 201 Created - event created. 400 Bad Request - invalid event data. |
| PUT | /api/events/{eventId} | Updates an existing event owned by the logged-in organiser | organizer | {name, description, eventDate, location, distanceKm, eventType, bannerImageUrl} | 200 OK - event updated. 403 Forbidden - Organiser does not own event. 404 Not Found - |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | event not found. |
| DELETE | /api/events/{eventId} | Deletes an event owned by the logged-in organiser | organizer | None | 200 OK - event returned. 404 Not Found - event does not exist. |
| GET | /api/events/{eventId}/catagories | Displays all catagories available for a selected event | Any(logged in) | None | 200 OK - Organiser's events returned. 403 Forbidden - incorrect role. |
| POST | /api/events/{eventId}/catagories | Adds a category to an event | organizer | { name, description, eventDate, location, distanceKm, eventType, bannerImageUrl } | 201 Created - event created. 400 Bad Request - invalid event data. |
| PUT | /api/events/{eventId}/catagories | Updates a category | organizer | { name, description, eventDate, location, distanceKm, eventType, bannerImageUrl } | 200 OK - event updated. 403 Forbidden - Organiser does not own event. 404 Not Found - event not found. |
| POST | /api/events/{eventId}/enrolments | Enrols the logged-in | Participant | None | 201 Created - enrolment created. |

| | | | | | |
|---|---|---|---|---|---|
| | | Participant into an event using a selected category. | | | 404 Not Found - event/category not found. 409 Conflict - Participant already enrolled. |
| GET | /api/participant/enrolments | Displays all event enrolments belonging to the logged-in Participant. | Participant | None | 200 OK - enrolments returned. |
| PATCH | /api/enrolments/{enrolmentId}/cancel | Cancels the logged-in Participant's enrolment. | Participant | None | 200 OK - enrolment cancelled. 404 Not Found - enrolment not found. |
| GET | /api/events/{eventId}/enrolments | Displays all Participants enrolled in an Organiser's event. | Organiser | None | 200 OK - enrolments returned. 403 Forbidden - Organiser does not own event. |
| POST | /api/enrolments/{enrolmentId}/result | Records a Participant's finishing | Organiser | { finishTime, finishingPosition } | 201 Created - result recorded. 400 Bad |

| | | | | | |
|---|---|---|---|---|---|
| | | time and finishing position. | | | Request - invalid result data. 404 Not Found - enrolment not found. |
| PUT | /api/enrolments/{enrolmentId}/result | Updates an existing Participant result. | Organiser | { finishTime, finishingPosition } | 200 OK - result updated. 400 Bad Request - invalid result data. 404 Not Found - result not found. |
| GET | /api/events/{eventId}/results | Displays all results for an event owned by the logged-in Organiser. | organizer | none | 200 OK - event results returned. 403 Forbidden - Organiser does not own event. |

# ERD DIAGRAM:


prog 2b ERD.pdf
File C:/Users/Mamello%20S%20PC/Downloads/prog%202b%20ERD.pdf
Draw
Ask Copilot
1
of 1
Edit with Acrobat
Chat
User
+int UserId PK
+string FullName
+string Email UK
+string PasswordHash
+string Role
+string ProfileImageUrl
+datetime CreatedAt
+bool IsActive
has
Organiser
+int OrganiserId PK FK
+string OrganisationName
+string PhoneNumber
creates
Event
+int EventId PK
+int OrganiserId FK
+string Name
+string Description
+datetime EventDate
+string Location
+decimal DistanceKm
+string EventType
+string BannerImageUrl
+datetime CreatedAt
has
Participant
+int ParticipantId PK FK
+date DateOfBirth
+string PhoneNumber
+string EmergencyContact
makes
receives
contains
Category
+int CategoryId PK
+int EventId FK
+string Name
+string CategoryType
+int MinAge
+int MaxAge
+decimal DistanceKm
+string Description
selected for
EventEnrolment
+int EnrolmentId PK
+int ParticipantId FK
+int EventId FK
+int CategoryId FK
+datetime EnrolledAt
+string Status
produces
Result
+int ResultId PK
+int EnrolmentId FK
+int FinishTimeSeconds
+int FinishingPosition
+datetime RecordedAt
MermaidEditor.io

# SQL CODE:

```
/* ============================================================
  RaceDay Database
  PROG6212 POE - Part 1
  SQL Server / SSMS
  ============================================================ */


------------------------------------------------------------
-- 1. CREATE DATABASE
------------------------------------------------------------

IF DB_ID('RaceDayDB') IS NULL
BEGIN
   CREATE DATABASE RaceDayDB;
END;
GO

USE RaceDayDB;
GO



------------------------------------------------------------
-- 2. DROP EXISTING TABLES
-- Allows the script to be run again during testing
------------------------------------------------------------
```

```
DROP TABLE IF EXISTS Results;
DROP TABLE IF EXISTS EventEnrolments;
DROP TABLE IF EXISTS Categories;
DROP TABLE IF EXISTS Events;
DROP TABLE IF EXISTS Participants;
DROP TABLE IF EXISTS Organisers;
DROP TABLE IF EXISTS Users;
GO


------------------------------------------------------------
-- 3. USERS TABLE
------------------------------------------------------------

CREATE TABLE Users
(
    UserId INT IDENTITY(1,1) PRIMARY KEY,

    FullName NVARCHAR(120) NOT NULL,

    Email NVARCHAR(255) NOT NULL UNIQUE,

    PasswordHash NVARCHAR(255) NOT NULL,

    Role NVARCHAR(20) NOT NULL,
```

```
    ProfileImageUrl NVARCHAR(500) NULL,

    CreatedAt DATETIME2 NOT NULL
        DEFAULT GETDATE(),

    IsActive BIT NOT NULL
        DEFAULT 1,

    CONSTRAINT CK_Users_Role
        CHECK (Role IN ('Organiser', 'Participant'))
);
GO



------------------------------------------------------------
-- 4. ORGANISERS TABLE
------------------------------------------------------------

CREATE TABLE Organisers
(
    OrganiserId INT PRIMARY KEY,

    OrganisationName NVARCHAR(150) NULL,

    PhoneNumber NVARCHAR(30) NULL,
```

```
    CONSTRAINT FK_Organisers_Users
        FOREIGN KEY (OrganiserId)
        REFERENCES Users(UserId)
        ON DELETE CASCADE
);
GO


------------------------------------------------------------
-- 5. PARTICIPANTS TABLE
------------------------------------------------------------

CREATE TABLE Participants
(
    ParticipantId INT PRIMARY KEY,

    DateOfBirth DATE NULL,

    PhoneNumber NVARCHAR(30) NULL,

    EmergencyContact NVARCHAR(120) NULL,

    CONSTRAINT FK_Participants_Users
        FOREIGN KEY (ParticipantId)
        REFERENCES Users(UserId)
```

```
    ON DELETE CASCADE
);
GO


------------------------------------------------------------
-- 6. EVENTS TABLE
------------------------------------------------------------

CREATE TABLE Events
(
  EventId INT IDENTITY(1,1) PRIMARY KEY,

  OrganiserId INT NOT NULL,

  Name NVARCHAR(150) NOT NULL,

  Description NVARCHAR(1000) NOT NULL,

  EventDate DATETIME2 NOT NULL,

  Location NVARCHAR(250) NOT NULL,

  DistanceKm DECIMAL(6,2) NOT NULL,

  EventType NVARCHAR(10) NOT NULL,
```

```
    BannerImageUrl NVARCHAR(500) NULL,

    CreatedAt DATETIME2 NOT NULL
        DEFAULT GETDATE(),

    CONSTRAINT FK_Events_Organisers
        FOREIGN KEY (OrganiserId)
        REFERENCES Organisers(OrganiserId),

    CONSTRAINT CK_Events_Distance
        CHECK (DistanceKm > 0),

    CONSTRAINT CK_Events_Type
        CHECK (EventType IN ('Run', 'Walk', 'Cycle'))
);
GO



------------------------------------------------------------
-- 7. CATEGORIES TABLE
------------------------------------------------------------

CREATE TABLE Categories
(
    CategoryId INT IDENTITY(1,1) PRIMARY KEY,
```

```
EventId INT NOT NULL,

Name NVARCHAR(100) NOT NULL,

CategoryType NVARCHAR(20) NOT NULL,

MinAge INT NULL,

MaxAge INT NULL,

DistanceKm DECIMAL(6,2) NULL,

Description NVARCHAR(300) NULL,

CONSTRAINT FK_Categories_Events
  FOREIGN KEY (EventId)
  REFERENCES Events(EventId),

CONSTRAINT UQ_Categories_Event_Name
  UNIQUE (EventId, Name),

CONSTRAINT UQ_Categories_Category_Event
  UNIQUE (CategoryId, EventId),

CONSTRAINT CK_Categories_Type
```

```
  CHECK (CategoryType IN ('Age', 'Distance')),

CONSTRAINT CK_Categories_Age
  CHECK
  (
    (MinAge IS NULL OR MinAge >= 0)
    AND
    (MaxAge IS NULL OR MaxAge >= 0)
    AND
    (
      MinAge IS NULL
      OR MaxAge IS NULL
      OR MaxAge >= MinAge
    )
  ),

CONSTRAINT CK_Categories_Distance
  CHECK
  (
    DistanceKm IS NULL
    OR DistanceKm > 0
  ),

CONSTRAINT CK_Categories_TypeFields
  CHECK
  (
```

```
        (
            CategoryType = 'Age'
            AND DistanceKm IS NULL
            AND (MinAge IS NOT NULL OR MaxAge IS NOT NULL)
        )

        OR

        (
            CategoryType = 'Distance'
            AND DistanceKm IS NOT NULL
            AND MinAge IS NULL
            AND MaxAge IS NULL
        )
    )
);
GO


------------------------------------------------------------
-- 8. EVENT ENROLMENTS TABLE
------------------------------------------------------------

CREATE TABLE EventEnrolments
(
    EnrolmentId INT IDENTITY(1,1) PRIMARY KEY,
```

```
ParticipantId INT NOT NULL,

EventId INT NOT NULL,

CategoryId INT NOT NULL,

EnrolledAt DATETIME2 NOT NULL
  DEFAULT GETDATE(),

Status NVARCHAR(20) NOT NULL
  DEFAULT 'Confirmed',

CONSTRAINT FK_EventEnrolments_Participants
  FOREIGN KEY (ParticipantId)
  REFERENCES Participants(ParticipantId),

CONSTRAINT FK_EventEnrolments_Events
  FOREIGN KEY (EventId)
  REFERENCES Events(EventId),

CONSTRAINT FK_EventEnrolments_CategoryEvent
  FOREIGN KEY (CategoryId, EventId)
  REFERENCES Categories(CategoryId, EventId),

CONSTRAINT UQ_EventEnrolments_Participant_Event
```

```
    UNIQUE (ParticipantId, EventId),

  CONSTRAINT CK_EventEnrolments_Status
    CHECK
    (
      Status IN ('Pending', 'Confirmed', 'Cancelled')
    )
);
GO



------------------------------------------------------------
-- 9. RESULTS TABLE
------------------------------------------------------------

CREATE TABLE Results
(
  ResultId INT IDENTITY(1,1) PRIMARY KEY,

  EnrolmentId INT NOT NULL UNIQUE,

  FinishTimeSeconds INT NOT NULL,

  FinishingPosition INT NOT NULL,

  RecordedAt DATETIME2 NOT NULL
```

```
    DEFAULT GETDATE(),

  CONSTRAINT FK_Results_EventEnrolments
    FOREIGN KEY (EnrolmentId)
    REFERENCES EventEnrolments(EnrolmentId)
    ON DELETE CASCADE,

  CONSTRAINT CK_Results_FinishTime
    CHECK (FinishTimeSeconds > 0),

  CONSTRAINT CK_Results_Position
    CHECK (FinishingPosition > 0)
);
GO


/* ============================================================
  10. SAMPLE DATA
  ============================================================ */


------------------------------------------------------------
-- USERS
------------------------------------------------------------

INSERT INTO Users
(
```

```
    FullName,
    Email,
    PasswordHash,
    Role
)
VALUES
(
    'Thabo Mokoena',
    'thabo.organiser@raceday.co.za',
    '$2b$12$DemoPasswordHashOrganiser1',
    'Organiser'
),
(
    'Naledi Jacobs',
    'naledi.organiser@raceday.co.za',
    '$2b$12$DemoPasswordHashOrganiser2',
    'Organiser'
),
(
    'Lethabo Molefe',
    'lethabo.participant@raceday.co.za',
    '$2b$12$DemoPasswordHashParticipant1',
    'Participant'
),
(
    'Aisha Khan',
```

```
    'aisha.participant@raceday.co.za',
    '$2b$12$DemoPasswordHashParticipant2',
    'Participant'
);
GO


------------------------------------------------------------
-- ORGANISERS
------------------------------------------------------------

INSERT INTO Organisers
(
    OrganiserId,
    OrganisationName,
    PhoneNumber
)
VALUES
(
    1,
    'Jozi Road Events',
    '+27 11 555 0101'
),
(
    2,
    'Coastal Motion Events',
```

```
    '+27 31 555 0102'
);
GO



------------------------------------------------------------
-- PARTICIPANTS
------------------------------------------------------------

INSERT INTO Participants
(
    ParticipantId,
    DateOfBirth,
    PhoneNumber,
    EmergencyContact
)
VALUES
(
    3,
    '2008-03-12',
    '+27 71 555 0201',
    'Mpho Molefe - +27 72 555 1201'
),
(
    4,
    '1994-07-25',
```

```
    '+27 72 555 0202',
    'Imran Khan - +27 73 555 1202'
);
GO



------------------------------------------------------------
-- EVENTS
------------------------------------------------------------

INSERT INTO Events
(
    OrganiserId,
    Name,
    Description,
    EventDate,
    Location,
    DistanceKm,
    EventType
)
VALUES
(
    1,
    'Johannesburg Spring Run',
    'A timed road running event in Johannesburg.',
    '2026-10-17 07:00:00',
```

```
    'Johannesburg, Gauteng',
    10.00,
    'Run'
),
(
    2,
    'Durban Coastal Cycle',
    'A coastal cycling event in Durban.',
    '2026-10-24 06:30:00',
    'Durban, KwaZulu-Natal',
    40.00,
    'Cycle'
),
(
    1,
    'Pretoria Heritage Walk',
    'A community walking event through Pretoria.',
    '2026-11-07 08:00:00',
    'Pretoria, Gauteng',
    5.00,
    'Walk'
);
GO
```

---

```
-- CATEGORIES
-------------------------------------------------------------


INSERT INTO Categories
(
   EventId,
   Name,
   CategoryType,
   MinAge,
   MaxAge,
   DistanceKm,
   Description
)
VALUES
(
   1,
   'Under 20',
   'Age',
   0,
   19,
   NULL,
   'Participants younger than 20.'
),
(
   1,
   'Open',
```

```
    'Age',
    20,
    59,
    NULL,
    'Participants aged between 20 and 59.'
),
(
    1,
    'Senior',
    'Age',
    60,
    NULL,
    NULL,
    'Participants aged 60 and older.'
),

(
    2,
    '20 km',
    'Distance',
    NULL,
    NULL,
    20.00,
    '20 kilometre cycling category.'
),
(
```

```
    2,
    '40 km',
    'Distance',
    NULL,
    NULL,
    40.00,
    '40 kilometre cycling category.'
),

(
    3,
    'Under 18',
    'Age',
    0,
    17,
    NULL,
    'Participants younger than 18.'
),
(
    3,
    'Open 18+',
    'Age',
    18,
    NULL,
    NULL,
    'Participants aged 18 and older.'
```

```
);
GO



------------------------------------------------------------
-- EVENT ENROLMENTS
------------------------------------------------------------

INSERT INTO EventEnrolments
(
   ParticipantId,
   EventId,
   CategoryId,
   Status
)
VALUES
(
   3,
   1,
   1,
   'Confirmed'
),
(
   4,
   1,
   2,
```

```
    'Confirmed'
),
(
    4,
    2,
    5,
    'Confirmed'
),
(
    3,
    3,
    7,
    'Confirmed'
);
GO



------------------------------------------------------------
-- RESULTS
------------------------------------------------------------

INSERT INTO Results
(
    EnrolmentId,
    FinishTimeSeconds,
    FinishingPosition
```

```
)
VALUES
(
  1,
  3124,
  1
),
(
  2,
  3478,
  2
);
GO


/* ============================================================
  11. VERIFY DATA
  ============================================================ */

SELECT * FROM Users;

SELECT * FROM Organisers;

SELECT * FROM Participants;

SELECT * FROM Events;
```

```
SELECT * FROM Categories;

SELECT * FROM EventEnrolments;

SELECT * FROM Results;
GO
```

# REFERENCES: